

算法与系统协同设计：训练、推理与 Agent

目 录

1 引言：为什么我们需要算法与系统协同设计	3
1.1 Token 经济：吞吐才是新的货币	3
1.2 Scaling Law 与数据飞轮	3
1.3 效率的四要素	4
1.4 MLSys 三角：算力、存储与通信	4
1.5 Agent 与 Harness：大模型的运行外壳	4
2 第一章：训练系统	5
2.1 数据流水线	5
2.2 优化器：从 Adam 到 Muon	5
2.3 架构：MoE 与混合注意力	6
2.3.1 MoE：以稀疏换效率	6
2.3.2 混合注意力架构	6
2.4 并行策略	6
2.5 参数高效微调：LoRA 与 QLoRA	7
2.6 训练框架对比	7
2.7 后训练：SFT、RL 与 OPD	8
2.8 可扩展 RL 基础设施	9
2.9 训练与推理基础设施的对比	9
2.10 Agentic RL 基础设施	10
3 第二章：推理优化	10
3.1 推理的两阶段瓶颈：Prefill 与 Decode	10
3.2 优化的五个方向	11
3.3 低比特量化	11
3.3.1 量化基础	11
3.3.2 NVFP4：训练与推理的 4 位浮点	12
3.3.3 异常值问题	13
3.4 高效注意力	14
3.4.1 三大注意力家族	14
3.4.2 FlashAttention：IO 感知的精确注意力	14
3.4.3 Ring Attention	15
3.4.4 线性注意力	15
3.4.5 稀疏注意力	16
3.5 KV Cache 压缩	17

3.5.1 问题规模	17
3.5.2 KV Cache 压缩分类	17
3.5.3 三种 KV Cache 压缩方法	17
3.5.4 分层 KV Cache 系统	18
3.5.5 Hy-Memory 案例	19
3.6 KV Cache 与注意力架构协同设计	19
3.6.1 面向世界模型的注意力: WorldAttention	20
3.7 高效解码	20
3.7.1 投机解码	20
3.7.2 多 Token 预测 (MTP)	21
3.7.3 Diffusion-LLM 的 MTP	21
3.7.4 K-Forcing	21
3.8 推理框架	22
3.9 分离部署	23
3.10 多模态与视频生成推理加速	23
3.10.1 理解方向 (Understanding)	23
3.10.2 生成方向 (Generation)	23
3.10.3 视频生成基础设施	23
3.10.4 Block Diffusion 与 Inferix	23
4 第三章: Agent Loop	24
4.1 什么是 Agent Loop	24
4.2 AutoResearch: 自然语言驱动的研究闭环	24
4.3 启发式学习与持续学习	24
4.4 自我改进的 Harness	25
4.5 Agent-as-a-Router	25
4.6 Kernel 优化 Agent	26
5 第四章: 前沿与展望	26
5.1 多模态基础模型的演进	26
5.2 从 VLA 到世界动作模型	26
5.3 4D 世界模型	27
5.4 Agentic 世界模型	27
5.5 产业级基础设施挑战	27
5.5.1 训练侧	27
5.5.2 推理侧	27
5.6 面向 Agent 的 AI 芯片	28
6 本章你将学会	28
7 要点速查	28
8 小结	30

1 引言：为什么我们需要算法与系统协同设计

我们从一张时间线开始。2012 年 **AlexNet** 在 ImageNet 上一战成名，深度学习从此进入工业界。此后几乎每年都有一次架构跃迁：2014 年 **VGGNet** 把网络做深，2015 年 **ResNet** 用残差连接解决梯度消失，2017 年 **Transformer** 用自注意力取代卷积成为新的通用架构，2018 年 **BERT** 让预训练加微调成为自然语言处理的标准范式，2020 年 **ViT** 把 Transformer 引入视觉。到了 2022 年 **ChatGPT** 引爆大模型时代，2024 年 **DeepSeek-V3** 以开源和高效震惊业界，2026 年我们迎来了 **Claude Fable 5** 与 **GPT-5.6** 这一代具备 Agent 能力的模型。

直觉 你不妨把这条时间线想成两条暗线的赛跑：一条是模型架构的演进，从 CNN 到 Transformer 再到 MoE；另一条是规模的爆炸，从 1.5 亿参数的 GPT-2 到千亿乃至万亿参数。每一次跃迁都不是孤立的事件，而是算法创新和基础设施能力共同到达临界点的结果。

1.1 Token 经济：吞吐才是新的货币

2026 年的行业现实是：所有人都在 Token Mxing（最大化 Token 消耗）。Google I/O 2026 报告其月处理量达到 **3.2 千万亿** (3.2 Quadrillion) tokens；OpenAI 的 API 吞吐量约为每分钟 150 亿 tokens（折合每月约 21.6 万亿），自 2023 年以来增长了 50 倍。OpenAI 单一最大企业客户每月消耗 1000 亿 tokens。从 2026 到 2030 年，行业 Agent 加 LLM 的 Token 总量预计从每月 1.5 千万亿增长到 120 千万亿，24 倍的增长。

这意味着什么？模型数量不再是竞争力的核心指标，**吞吐** (throughput) 和 **Token 效率** (token productivity) 才是新的护城河。谁用最少的算力处理最多的 Token，谁就赢了。

1.2 Scaling Law 与数据飞轮

大模型有三条交织的 Scaling Law（缩放定律），它们共同构成了加速增长的飞轮：

1. **预训练 Scaling Law**：训练算力（FLOPs）与测试 loss 之间存在幂律关系。从 GPT-2 (1.5B, 2019) 到 GPT-4.5 / Grok 3 (2025)，计算量跨越 10^{21} 到 10^{26} FLOPs，loss 在对数坐标上持续下降。但到 2024 至 2025 年，高质量网络数据几乎耗尽，收益开始转向合成数据。
2. **测试时 Scaling Law**：推理时投入更多思考 Token 可以提升效果。**o1** (2024.09) 在 AIME 2024 上的准确率与测试时算力的对数成正比；**o3** 在 ARC-AGI 上从 75.7% 提升到 87.5% 用了 172 倍算力；**DeepSeek-R1** 则证明了长思维链可以从纯 RL 中自然涌现。
3. **自演化数据飞轮**：更多 Agent 交互产生更多数据，更多数据训练出更好的模型，更好的模型部署为更强的 Agent，再产生更多数据。预训练、测试时缩放和数据飞轮三者复合，形成加速增长。

例 把飞轮想成一个正反馈循环：Agent 部署到真实任务后，用户的交互、反馈和自我博弈都变成新的训练数据。这些数据回流到模型训练中，下一代模型更强，部署后又能处理更复杂的任务、产生更高质量的数据。每一轮迭代都在上一轮的基础上加速。

1.3 效率的四要素

面对如此庞大的算力需求，效率优化可以从多个维度入手：

- 1. **数据压缩** (Data compression)：例如分词器 (Tokenizer) 的设计，数据重标注与过滤。
- 2. **硬件与算法协同设计** (Hardware-algorithm co-design)：例如 **FlashAttention** 通过分块计算减少 HBM 访存，**DeepSeek-V4** 在 FP4 精度下训练。
- 3. **分布式计算** (Distributed computing)：例如 **FSDP**、**Megatron-LM** 把模型切分到数千张 GPU 上。
- 4. **基础设施与 AI 加速器** (Infra, AI accelerator)：例如 **vLLM**、**VeRL**、**Groq** 等推理与服务系统。
- 5. **缩放定律** (Scaling law)：理解 LLM 本质上是一个压缩器，更大的压缩比意味着更好的泛化。
- 6. **高效架构与模块** (Efficient architecture and module)：例如 **Mamba**、**MoE** 等新型结构。
- 7. **模型压缩** (Model compression)：例如剪枝、量化、蒸馏。

1.4 MLSys 三角：算力、存储与通信

维度	含义	典型瓶颈
算力 (COMPUTE)	GPU/TPU 的 FLOPs	大矩阵乘法 GEMM
存储 (MEMORY)	SRAM/DRAM/SSD, KV cache, 检查点	HBM 容量与带宽跟不上算力增长
通信 (COMMUNICATION)	PCIe, NVLink, InfiniBand	集合通信延迟, 跨节点带宽

直觉 这三个维度就像一个三角形的三条边，任何一条边短了，整个系统就被卡住。算力再强，如果 HBM 带宽跟不上，GPU 只能空转等数据；通信再快，如果存储装不下模型，就无法训练。HBM/KV cache 的 I/O 必须跟得上算力，互连带宽必须匹配 FLOPs，而分片存储用网络交换内存 (ZeRO 的核心思想)。三者必须协同扩展。

1.5 Agent 与 Harness：大模型的运行外壳

在 Agent 时代，一个重要的等式是：

Agent = Model + Harness (1.1)

Harness (外壳/框架) 是一段代码，它编排提示词、工具调用、子 Agent、控制流、记忆和工作流逻辑如何协同工作。代码是定义程序和系统的通用语言。主流编码 Agent (如 **Claude Code**、**Codex**、**OpenCode**、**Cursor**) 的核心接口就是一个 Harness。

直觉 为什么要把 Model 和 Harness 分开？因为它们的更新代价天差地别。Harness 层面的优化是白盒的、快速且廉价的，改几行代码就能生效；模型层面的更新是黑盒的、慢且昂贵的，需要收集数据、训练、评估。这就是**双层优化** (Bi-level Optimization)：System

1（实时控制）用 Harness 快速迭代，System 2（模型改进）用梯度更新缓慢但根本地提升能力。

本章接下来的内容将沿着这条主线展开：先讲训练系统如何把大模型训练出来，再讲推理系统如何高效地部署模型，然后讲 Agent Loop 如何让模型在真实任务中自我迭代，最后展望未来方向。

2 第一章：训练系统

训练一个大模型是一场数据、算法与基础设施的协同设计。我们从数据流水线开始，一路讲到优化器、架构选择、并行策略、微调方法、训练框架，最后到后训练和 RL 基础设施。

2.1 数据流水线

预训练数据的质量直接决定模型的上限。前沿实验室的数据流水线通常包含以下步骤：

1. **原始来源**（Raw sources）：网页、代码、书籍、论文。
2. **去重**（Dedup）：用 **MinHash** 加精确匹配去除重复文档。
3. **质量过滤**（Quality filter）：用模型分类器筛选高质量文本。
4. **去污染**（Decontaminate）：剥离混入的基准测试数据，防止考试作弊。
5. **合成生成**（Synthetic gen）：用 LLM 重写、生成教科书内容、问答对和思维链。
6. **混合与课程**（Mix & curriculum）：确定各领域权重和训练阶段。

一个惊人的事实：前沿预训练数据中，合成数据已经超过 90%，精选真实数据不足 10%。数据流水线本身已经是模型在环（*model-in-the-loop*）的，LLM 负责过滤、重写和生成大部分训练材料。数据效率、数据质量、数据多样性和数据数量都至关重要，而分词器（Tokenizer）的选择对效率有深远影响。

2.2 优化器：从 Adam 到 Muon

AdamW 是自 GPT-3 以来深度学习的事实标准优化器。它为每个参数维护一阶动量 m 和二阶动量 v ，用它们计算自适应的步长：

$$m \leftarrow \beta_1 m + (1 - \beta_1)g \quad (2.1)$$

$$v \leftarrow \beta_2 v + (1 - \beta_2)g^2 \quad (2.2)$$

$$\theta \leftarrow \theta - \eta \cdot \frac{m}{\sqrt{v} + \varepsilon} \quad (2.3)$$

其中 β_1, β_2 是动量衰减系数， η 是学习率， ε 防止除零，加上解耦的权重衰减。AdamW 在各种规模上都很鲁棒，是默认选择。

直觉 Adam 的核心思想是：每个参数应该有自己的学习率。梯度经常很大的参数，它的二阶动量 v 也大，所以分母大，步长被缩小；梯度经常很小的参数， v 小，步长相对放大。这是一种快的地方走慢点、慢的地方走快点的自适应策略。

但 Adam 有一个被忽视的缺陷：它把参数矩阵的更新当作独立的标量来处理，完全忽略了矩阵结构。**Muon** (Matrix-aware Unified Optimizer) 对此做了改进：它用 **Newton-Schulz 迭代**对动量更新做正交化。

直觉 什么是正交化？想象你的更新方向是一个矩阵，正交化后的矩阵保持了方向信息但去除了冗余的缩放。直觉上，正交矩阵的奇异值都是 1，这意味着更新在每个方向上的力度是均匀的，不会在某些方向过度更新而在另一些方向欠更新。

例 Muon 的实测效果：达到 AdamW 级别的 loss 只需约 52% 的训练 FLOPs，即约 2 倍的计算效率。**MuonClip** 在 **Kimi K2** (1 万亿参数 MoE, 15.5 万亿 tokens) 上实现了零 loss spike 的稳定训练。一句话：更好的优化器就是免费的算力。

2.3 架构：MoE 与混合注意力

2.3.1 MoE：以稀疏换效率

MoE (Mixture of Experts, 混合专家) 的核心思想是：模型总参数量很大，但每个 token 只激活其中一小部分专家。以 **DeepSeek-V3** 为例，它有 6710 亿参数，但每个 token 只激活 370 亿参数。这样可以在不增加单 token 计算量的情况下大幅扩展模型容量。

直觉 你可以把 MoE 想象成一个大医院：医院有几百个专科医生（专家），但每个病人（token）只需要看其中几个科室。医院的整体规模很大（参数多），但每个病人的就诊时间不长（计算量小）。路由器（Router）就是分诊台，它决定每个 token 去找哪几个专家。

2.3.2 混合注意力架构

国内前沿模型普遍采用混合注意力策略来应对长上下文：

1. **全注意力加稀疏注意力**：例如 SWA (Sliding Window Attention, 滑动窗口注意力) 和 DSA (DeepSeek Sparse Attention, DeepSeek 稀疏注意力)。在部分层使用全注意力保证精度，其余层使用稀疏注意力降低开销。
2. **全注意力加线性注意力**：例如 GDN (Gated DeltaNet) 和 KDA。线性注意力层用 $O(N)$ 复杂度处理长序列，全注意力层保留精确回忆能力。

混合架构的本质是好钢用在刀刃上：在需要精确回忆的层用全注意力，在可以容忍近似的长上下文层用稀疏或线性注意力。这样既控制了内存和计算开销，又不会严重损失效果。

2.4 并行策略

当模型大到单卡放不下时，必须把计算切分到多张 GPU 上。以下五种并行策略各有优缺点，前沿训练通常是它们的组合（4D 甚至 5D 并行）。

并行策略	切分对象	优点	缺点
数据并行 (DP/ ZeRO/FSDP)	batch 切到各 GPU； ZeRO/FSDP 进一步 切分参数、梯度、优 化器状态	简单；吞吐近线性扩 展；无需改模型代码	显存冗余（除非分片）； all-reduce 通信量随模 型增大

张量并行 (TP)	层内权重矩阵	可训单卡放不下的大层；降低单卡显存与延迟	每层都要通信，需 NVLink 级带宽；一般限节点内
流水线并行 (PP)	按层切成多个 stage	通信量小；可跨节点扩展深度	流水线气泡；需 micro-batch 调度 (1F1B、interleaved)；stage 负载均衡难
专家并行 (EP)	MoE 专家分布到各 GPU	参数规模扩展而 FLOPs/token 基本不变；天然适配 MoE	all-to-all 路由通信；专家负载不均
序列/上下文并行 (CP)	序列维度 (Ring/Ulysses attention)	支持百万 token 上下文；切分激活显存	额外 attention 通信；KV 交换复杂

直觉 五种策略切分的是模型的不同维度：DP 切 batch，TP 切层内矩阵，PP 切层间，EP 切专家，CP 切序列。它们是正交的，可以组合使用。前沿训练通常组合 DP × TP × PP × EP (× CP)，根据互连拓扑精心调优，这就是所谓的 4D/5D 并行。

2.5 参数高效微调：LoRA 与 QLoRA

全量微调一个大模型需要巨大的显存。**LoRA** (Low-Rank Adaptation, 低秩适配) 的洞察是：微调时的权重更新 ΔW 是低秩的，可以分解为两个小矩阵的乘积。

$$h = W_0 x + B A x, \quad r \ll d \quad (2.4)$$

其中 W_0 是冻结的预训练权重， $A \in R^{r \times d}$ 和 $B \in R^{d \times r}$ 是新引入的可训练矩阵，秩 r 远小于维度 d 。训练时只更新 A 和 B ，参数量减少可达 10000 倍，GPU 显存减少 3 倍。训练完成后可以把 $W_0 + BA$ 合并，推理时零额外开销。

例 以 GPT-3 175B 为例，LoRA 可以将可训练参数减少 10000 倍，GPU 显存减少 3 倍，而效果接近全量微调。**QLoRA** 更进一步：在 4 位量化的基座模型上做 LoRA，可以在单张 48GB GPU 上微调 650 亿参数的模型。核心原则是：用不到 1% 的权重来适配大模型，以极低成本获得接近全量微调的质量。

LoRA 之所以有效，是因为微调任务（如指令跟随、领域适配）不需要改变模型的全部能力，只需要在一个低维子空间内做调整。这和矩阵的低秩近似思想一致：大部分信息集中在少数几个主方向上。

2.6 训练框架对比

框架	核心思路	优点	缺点
PyTorch FSDP2	原生全分片：参数/梯度/优化器状态	PyTorch 原生；与 torch.compile、TP、激	无内置流水线并行；fused kernel 少；超大规模需自行调优

	(ZeRO-3 式, 基于 DTensor)	活重算可组合; 上手容易	
Megatron-LM / Megatron Core	NVIDIA 全栈: TP/PP/DP/EP/CP 加 fused kernel 加 FP8 Transformer Engine	前沿规模 MFU 最高; 100B-1T 模型久经考验 (Nemotron、GLM 等)	学习曲线陡, 代码复杂; 绑定 NVIDIA 生态
DeepSpeed	ZeRO-1/2/3 加 CPU/NVMe offload, MoE 与流水线引擎	功能全; 靠 offload 用小集群训大模型; 与 HF 集成顺滑	迭代放缓; 配置繁杂; 前沿规模性能落后 Megatron

选择建议: 追求极致规模和 MFU 用 Megatron; 追求 PyTorch 原生灵活性和可组合性用 FSDP/TorchTitan; 显存极其紧张时用 DeepSpeed 的 offload 能力。没有万能最快的框架, 只有最适合你的模型结构、硬件和目标的框架。

2.7 后训练: SFT、RL 与 OPD

预训练之后的后训练决定了模型能否真正有用。前沿实验室的后训练工作量大致分配为: SFT 约 70%, RL 约 20%, OPD 约 10%。

方法	目的	数据来源	目标分布	训练信号	On-policy?	遗忘/成本
SFT	注入新知识与格式, 快速建立基础能力	外部数据集	固定的外部数据集, 把模型拉向它	稠密: 逐 token 交叉熵 (Forward KL)	否	高/低
Offline RL (DPO)	用固定偏好数据做对齐, 简单便宜	固定的偏好数据集	偏好数据中的最优策略	成对偏好/隐式 reward	否	中/低
Online RL (PPO/GRPO)	在可验证任务上探索, 提升推理与 Agent 能力	当前 policy rollout	当前模型附近的 reward 最优策略	稀疏: 只有 outcome 级 reward (Reverse KL)	是	低/高
OPD	合并多个 teachers 的能力	学生采样轨迹	teacher 在学生自采样数据上的 logits	稠密: 逐 token 对齐 teacher (Reverse KL)	是	低/中

直觉 这四种方法的核心区别在于训练数据从哪来和目标分布是什么。SFT 把模型拉向外部数据集; Offline RL 把模型拉向偏好数据中的最优策略; Online RL 让模型在自己的采

样上探索，用环境的反馈做信号；OPD 则让 student 在自己采样的数据上对齐 teacher 的 logits。On-policy 数据是关键变量：在自己的样本上训练，更新始终在当前 policy 附近，可以防止灾难性遗忘。

SFT 用 *Forward KL*（把模型推向数据分布），RL 和 OPD 用 *Reverse KL*（让数据/teacher 分布被模型覆盖）。*Forward KL* 对低概率区域赋予高权重（*mode-covering*），*Reverse KL* 聚焦高概率区域（*mode-seeking*）。这就是为什么 RL 训练后的模型更锐利但可能丢掉一些长尾能力，而 SFT 后的模型更平滑。

2.8 可扩展 RL 基础设施

大模型的 RL 训练需要一个推理引擎来做 rollout（采样），一个训练引擎来更新参数，以及一个调度器来协调两者。以下是主流的 RL 训练系统：

系统	调度器	推理	训练	特点
VeRL (HybridFlow)	Ray	vLLM/ SGLang	FSDP/ Megatron	最大的生态
OpenRLHF	Ray	vLLM	DeepSpeed	最早在 RL 中采用 Ray 架构和 vLLM 推理加速
AReal (RealHF)	Custom	SGLang	Megatron	截断式 Rollout，纯异步 RL 更新
ROLL	Ray	vLLM/ SGLang	DeepSpeed/ Megatron	专注于 Agent 多轮 function calling 能力
Slime	Ray	SGLang	Megatron	极简，超 Lightweight
Nemo RL	Ray	vLLM	Megatron	NVIDIA 原汁原味

Slime 是极简主义的代表：它的公式就是 *Slime* = *Megatron* 训练 + *SGLang* rollout + 数据缓冲区，支持同步或全异步的 *Agentic RL*。它被用于 GLM-4 和 GLM-5 的训练。设计哲学是少即是多：用最少的组件实现最灵活的 RL 训练循环。

2.9 训练与推理基础设施的对比

训练和推理虽然都跑模型，但优化的目标截然不同：

- 训练**：目标是同步迭代更新参数。关键指标：吞吐（throughput）、MFU（Model FLOPs Utilization，算力利用率）、训练时长。训练引擎围绕算力加显存加通信，追求稳定的大批量参数更新。
- 推理**：目标是动态调度持续服务请求。关键指标：TTFT（Time To First Token，首 Token 延迟）、TPOT（Time Per Output Token，每 Token 延迟）、吞吐与并发。推理引擎围绕 Prefill 算力加 Decode 带宽加请求调度，在吞吐、延迟和并发之间权衡。

直觉 简单说：训练像一个工厂流水线，追求单位时间产出最多（吞吐）；推理像一个餐厅，追求每位客人快点上菜（延迟）同时服务尽可能多的桌（并发）。两者的瓶颈不同：训练瓶颈在算力和通信，推理瓶颈在访存和调度。

2.10 Agentic RL 基础设施

当 RL 训练的对象从做数学题变成完成多轮 Agent 任务时，基础设施面临全新的挑战：

维度	Agentic RL 基础设施
Rollout	多轮循环：LLM 与工具/环境交互，长时程交互，长尾严重
Reward	环境结果：测试通过、任务完成
执行方式	异步、解耦的 rollout 服务；训练和 rollout 分离；支持 partial rollout
瓶颈	长尾轨迹加海量并发工具调用拖停 GPU
环境	sandbox、浏览器、代码执行器：长生命周期、I/O 密集、需独立扩缩
权重同步	trainer 到推理引擎跨 GPU 池频繁同步

Agentic RL 本质上是一个分布式系统问题。最大的挑战是：训练引擎需要 GPU 做梯度更新，rollout 服务也需要 GPU 做推理，环境（sandbox、浏览器）是 I/O 密集的 CPU 任务。如果 rollout 产生了一条特别长的轨迹（比如 Agent 花了 1000 步才完成任务），训练引擎不能干等。解法是把 rollout 和训练解耦，支持 partial rollout（部分轨迹先送回训练），让 GPU 始终保持忙碌。

3 第二章：推理优化

推理（inference）是大模型服务用户的核心环节：训练只发生一次，推理却要在模型整个生命周期里重复万亿次，因此推理效率直接决定了服务成本和响应延迟。然而，一次推理请求并不是均匀的计算流，而是被自然地切成两个特性截然不同的阶段，理解这两阶段的差异是所有推理优化的出发点。

3.1 推理的两阶段瓶颈：Prefill 与 Decode

自回归 Transformer 的推理分为两个阶段：

- Prefill（预填充）阶段：**模型一次性读入并处理整个输入 prompt。所有输入 Token 之间互相做注意力，可以用大块矩阵乘法高度并行。GPU 算力被充分利用，因此这一阶段是**计算密集**（compute-bound）的：瓶颈在 FLOPs，提升方向是提高算力利用率。
- Decode（解码）阶段：**模型逐个生成输出 Token。每生成一个 Token，都要把整条历史 KV cache 重新读一遍、再追加一个新条目，却只做一次小规模的注意力和 FFN 计算。每步计算量很小，却要搬运大量显存，因此这一阶段是**访存密集**（memory-bound）的：瓶颈在 HBM 带宽，提升方向是减少访存、提高带宽利用率。

直觉 区分两个阶段的关键是**算术强度** (arithmetic intensity)，即每搬运一个字节的數據能做多少次浮点运算。Prefill 阶段参与运算的 Token 数 N 很大，矩阵乘法是大块的 $N \times d$ 运算，算术强度高，所以卡在算力上；Decode 阶段每步只新增一个 Token，却要和整个 KV cache 做点积，算术强度极低，GPU 大部分时间在等 HBM 把数据送来，所以卡在带宽上。这就是为什么同一块 GPU 在 Prefill 时算力打满，在 Decode 时算力却常常空转。

例 这种阶段差异直接影响硬件选型与部署。算力强但带宽相对弱的 GPU（如 H100）适合跑 Prefill，带宽富裕但算力弱的 GPU（如 H20）适合跑 Decode。长上下文 Agent 则会同时压垮两个阶段：Prefill 阶段要一次性处理几十万 Token（计算压力大），Decode 阶段每步都要重读膨胀的 KV cache（访存压力大）。本章后续的 PD 分离部署、KV cache 压缩、高效注意力等技术，本质上都是在分别缓解这两个阶段的瓶颈。

3.2 优化的五个方向

推理优化的核心原则是**算法与基础设施协同设计** (Algorithm and Infra Co-design)：既不单靠算法技巧，也不单靠硬件堆叠，而是让模型结构、数值格式和系统实现相互配合。具体从五个方向入手：

1. **低比特量化** (Low-bit Quantization)：用更少的位数表示权重和激活，直接降低显存占用、带宽和算力需求。
2. **高效注意力机制** (Efficient Attention Mechanism)：改造注意力的计算方式，降低 $O(N^2)$ 的计算复杂度和 KV cache 膨胀。
3. **稀疏化** (Sparsity)：只计算真正重要的部分，跳过冗余的 Token 或专家。
4. **高效解码范式** (Efficient Decoding Paradigm)：用投机解码、多 Token 预测等方法突破「每步一个 Token」的限制。
5. **蒸馏** (Distillation)：把大模型的能力压缩进小模型，降低单次推理成本。

本章将依次展开这些方向，并配合 KV cache 管理、推理框架、分离部署等系统级技术，说明它们如何协同作用。

3.3 低比特量化

3.3.1 量化基础

量化 (Quantization) 把高精度权重和激活映射到低比特表示（如 INT8/INT4 或 FP8/FP4），减少内存占用和带宽，同时让矩阵乘法在低精度硬件上高效运行。量化的收益是多方面的：更低功耗、更低存储、更低延迟、更低通信带宽。

量化的数学本质是一个仿射映射：把浮点值 x 通过缩放因子 s 和零点 z 映射到整数网格上的 x_q ，使用时再反量化回来：

$$x_q = \text{round}\left(\frac{x}{s} + z\right), \quad x \approx s \cdot (x_q - z) \quad (3.1)$$

对称量化 令 $z = 0$ ，网格关于原点对称，适合分布近似零均值的情况（如权重）；**非对称量化** 保留零点 z ，可以平移网格去覆盖偏置分布（如激活常偏向正值）。量化的位数 b 决定了网格点数 2^b 与表示范围：位数越低，点越少，量化误差 $x - s(x_q - z)$ 越大。

直觉 为什么权重量化比激活量化容易？因为权重在训练后是固定的，分布平稳且易于校准；而激活随输入变化，某些通道会出现远大于其他通道的异常值（outliers），把缩放因子 s 撑大，导致其余通道被量化到极少数几个网格点上，精度被严重稀释。这就是后文「异常值问题」的根源。

按量化的对象，量化还分为**仅权重量化**（weight-only，如 W8A16：权重 8 bit、激活 16 bit）和**权重激活联合量化**（如 W8A8：两者都 8 bit）。仅权重量化只省显存和权重搬运，矩阵乘法仍在高精度下做；联合量化才能让低精度硬件真正跑起高效的 GEMM，但对激活异常值更敏感。按校准时机，量化又分为**训练后量化**（PTQ, Post-Training Quantization，不重训，速度快但精度有损）和**量化感知训练**（QAT, Quantization-Aware Training，在训练中模拟量化误差，精度更高但成本大）。后文的 NVFP4 既可用于 PTQ（DeepSeek-R1 从 FP8 后训练量化到 NVFP4），也可用于 QAT（DeepSeek-V4 直接在 FP4 下训练）。

量化格式有三大类，它们在数值网络的分布方式上根本不同：

1. **二值/三值**（Binary/Ternary）：1 bit 只保留符号乘以一个缩放因子 α 。三值用 +1, 0, -1 三种值。极端压缩但精度损失大。
2. **定点整数**（Fixed-point INT-b）：均匀网格， 2^b 个等间距的值。可以无零点（对称量化）或有零点（非对称量化，用 z 平移网格）。
3. **浮点**（Floating-point FP4）：对数网格，在零附近密集（数值集中的区域），远离零处稀疏。配合两级缩放。

直觉 三种格式的本质区别在于数值网络怎么分布。INT 是均匀的，每隔固定距离放一个网格点；FP 是对数的，零附近密、远处疏。因为神经网络的权重和激活大多集中在零附近，所以 FP 格式在同样位数下通常比 INT 更准确。

例 以边缘设备为例，AQD（CVPR 2021 Oral）实现了纯整数网络：卷积、BN 和检测头全部用定点运算，推理时没有任何浮点操作，4 bit 的 FCOS 在 COCO 上匹配了全精度版本。FATNN（ICCV 2021）用三值 +1, 0, -1 把内积重设计为两个并行的位运算，复杂度比 2 bit 定点低 2 倍。这让低比特网络在边缘设备上部署而不损失精度。

3.3.2 NVFP4：训练与推理的 4 位浮点

NVFP4 是 NVIDIA 推出的 4 位浮点格式，采用**两级缩放**（two-level scaling）：每 16 个 FP4 值配一个 FP8 尺度（E4M3，精细的尺子），每个张量配一个 FP32 全局尺度（全局修剪）。FP4 采用 E2M1 格式，即 1 位符号加 2 位指数加 1 位尾数，指数以 1 为偏置，因此可表示值呈对数分布：

{0, plus.minus 0.5, plus.minus 1, plus.minus 1.5, plus.minus 2, plus.minus 3, plus.minus 4, plus.minus 6}, 共 15 个值。注意每翻一倍只插入一个中间值（如 2 与 4 之间有 3），这正是浮点「零附近密、远处疏」的由来；含两级缩放后每个值的有效位数约 4.5 bit。

直觉 两级缩放的实际计算是分层的：原始值 x 先除以全局 FP32 尺度 S_g 确定量程，再除以每 16 个值一组的 FP8 块尺度 S_l 做微调，落到 FP4 网格上；反量化时逐级乘回 S_l 与 S_g 。全局尺度像一把粗尺子量整体长度，块内 FP8 尺度像一把细尺子量每一小段。由于

16 个值共享一组粗刻度却各有精细缩放，4 bit 就能逼近 FP8 的精度。作为对比，MXFP4 用 32 值块配 E8M0（2 的整数次幂）尺度，粒度更粗，量化误差约高 88%，所以需要多 36% 的训练 Token 才能追上 NVFP4 的 loss。

实测结果令人震惊：

1. 在 Blackwell GPU 上，W4A4 GEMM（权重和激活都是 4 bit）的峰值推理吞吐是 FP8 的 3 倍，内存流量是 BF16 的 1/4。
2. 预训练方面，比 FP8 快 1.9 倍（Llama-3.1 405B 实测）。
3. DeepSeek-R1 从 FP8 后训练量化到 NVFP4，精度损失低于 1%（MMLU-Pro 85 → 84）。
4. MXFP4（32 值块，粗粒度缩放）需要多 36% 的训练 Token 才能匹配 NVFP4 的 loss。

NVFP4 已经被设计采纳：DeepSeek-V4 在 FP4（QAT）下训练其 1.6T MoE 专家权重；GPT-OSS 专家使用 MXFP4。生产栈是 TensorRT-LLM 加 TransformerEngine 的融合量化 GEMM 内核。两级缩放让 4 bit 几乎无损：以 FP8 级精度获得最高 3 倍吞吐。

3.3.3 异常值问题

当模型参数超过 67 亿时，激活的某些通道会出现**异常值**（outliers）：这些通道的激活值比其他通道大几十甚至上百倍。如果简单地把所有通道统一量化，异常值会撑大缩放因子，导致其他通道的精度严重下降。

直觉 异常值的根源在 Transformer 的结构：残差连接把各层激活累加到同一条残差流上，LayerNorm 后再经注意力与 FFN 投影，少数通道在训练中逐渐积累出远超平均的幅度。这些通道又对模型输出很关键（承载了大部分信息），不能简单截断。所以量化的难点不在「平均的值」，而在「既要保住异常值，又不让它撞垮其余通道」。

应对方案经历了从 2022 到 2025 年的演进，核心思路是把「难量化的激活」转化为「易量化的形式」：

1. **LLM.int8()**（2022）：混合精度，异常值通道保留 FP16，其余用 INT8。简单但无法做到更低位宽。
2. **SmoothQuant**（2023）：把激活的异常值平滑转移到权重上。具体地，对异常通道 j 引入缩放 s_j ，令 $x_j w_j = \left(\frac{x_j}{s_j}\right)(s_j w_j)$ ：把激活除以 s_j 压小，同时把对应权重列乘以 s_j 放大。矩阵乘法结果不变，但激活的异常值被「搬」到了分布更均匀的权重上，于是激活和权重都可以用 INT8 量化。
3. **QuaRot**（2024）：用随机正交矩阵旋转权重和激活，旋转后异常值被分散到所有维度，不再集中在少数通道。
4. **SpinQuant**（2024）：QuaRot 的升级版，用学习到的正交矩阵（而非随机矩阵）做旋转，效果更好。
5. **SageAttention / SageAttention2**（2025）：专门针对注意力计算的量化，用逐线程 INT4 量化进一步压缩。

直觉 旋转量化的数学原理是**旋转不变性**：注意力分数 QK^T 在正交变换 R 下保持不变，因为 $(QR)(R^T K^T) = Q(RR^T)K^T = QK^T$ 。所以我们可以选一个合适的 R （比如 Walsh-Hadamard 变换），把权重和激活都旋转，使得异常值被均匀分散，然后再量化。推理时只需要在输入端乘 R 、输出端乘 R^T ，开销很小。

3.4 高效注意力

3.4.1 三大注意力家族

注意力机制是 Transformer 的计算和内存瓶颈：标准 softmax 注意力的复杂度是 $O(N^2)$ ，长上下文下 KV cache 急剧膨胀。高效注意力有三大家族：

家族	复杂度	核心思想	代表模型
Softmax Attention	$O(N^2)$, IO 优化	精确注意力，分块计算不物化 $N \times N$ 矩阵	FlashAttention v1-v4
Linear Attention	$O(N)$ 时间, $O(1)$ 状态	用衰减状态替代 softmax, 压缩为 RNN 形式	Mamba, RWKV, GDN, Kimi Linear
Sparse Attention	$\approx O(N \cdot k)$	只关注重要的 Token, 可训练的块/Token 选择	DeepSeek DSA, MoBA, H2O

三者的权衡：Softmax 注意力效果最好但开销最大，KV cache 随上下文增长；线性注意力最省但精确回忆能力弱，所以前沿模型用混合架构（部分层 softmax 部分层 linear）；稀疏注意力在两者之间，用智能选择看哪些 Token 来逼近全注意力效果。

3.4.2 FlashAttention: IO 感知的精确注意力

FlashAttention 是近年来最重要的注意力优化之一。它的核心洞察是：标准注意力的瓶颈不在计算，而在**内存访问**。

标准注意力的三步是：

$$S = QK^T \rightarrow P = \text{softmax}(S) \rightarrow O = PV \quad (3.2)$$

中间矩阵 S 和 P 都是 $N \times N$ 的。在标准实现中， S 写入 HBM，再读出来算 softmax 写回 P ，再读 P 算 O 。 S 和 P 的读写量是 $O(N^2)$ ，远大于 Q, K, V 的 $O(Nd)$ （当 $N \gg d$ 时）。这意味着 GPU 的大部分时间花在 HBM 搬运数据上，而不是计算。

GPU 的内存层次如下：

层级	带宽	容量
SRAM（片上）	$\approx 19 \text{ TB/s}$	$\approx 20 \text{ MB}$
HBM（显存）	$\approx 1.5\text{-}2 \text{ TB/s}$	40 GB

两者带宽差约 10 倍。FlashAttention 的策略是**分块**（Tiling）：把 Q, K, V 切成小块，在 SRAM 内完成 $QK^T \rightarrow \text{softmax} \rightarrow \times V$ 的全部计算，中间结果不落盘（不写入 HBM），只写回最终的 O_i 。 $N \times N$ 的 S 和 P 从未完整写入 HBM，只有块大小的中间结果短暂存于 SRAM。

例 对比标准注意力和 FlashAttention:

指标	标准注意力	FlashAttention
计算量 (FLOPs)	$\Theta(N^2d)$	$\Theta(N^2d)$, 反向传播需重算
HBM 访问 (I/O)	$\Theta(Nd + N^2)$	$\Theta(N^2 \frac{d^2}{M})$, GPT-2 上减少 9 倍
额外内存	$O(N^2)$, 物化 $N \times N$ 分数矩阵	$O(N)$, 只保存每行 softmax 统计量
实测速度	1 倍 (PyTorch 基线)	7.6 倍注意力算子 (GPT-2); 3 倍训练; BERT MLPerf 提升 15%

关键: 计算量完全相同, 但 HBM 访问大幅减少。这就是 IO 感知设计的威力。

FlashAttention 的技术细节包括两个关键机制:

1. **分块合并丢弃** (Tile, merge, discard): 每次只加载一个 Q_i 块和一个 K_j, V_j 块到 SRAM, 在 SRAM 内算 $S_{i,j} = Q_i K_j^T$, 维护运行时统计量 (最大值 m_i 、归一化因子 ℓ_i 、累积输出 O_i), 处理完一块就合并到运行状态中, 中间分数块被丢弃。
2. **在线 softmax** (Online softmax via log-sum-exp): 不需要看到整行就能计算 softmax。用 log-sum-exp 技巧增量地合并每个 $\frac{K}{V}$ 块到每个 query 行。最终 $LSE_i = m_i + \log \ell_i$ 。

在线 softmax 是 FlashAttention 的数学核心。标准 softmax 需要先看到整行的最大值才能做数值稳定的归一化: $p_i = \frac{\exp(s_i - m)}{\sum \exp(s_j - m)} \exp(s_j - m)$ 。但在分块计算中看不到整行。解法是为每个 query 行维护运行状态三元组 (m, ℓ, O) : 最大值 m 、归一化因子 ℓ 、累积输出 O 。处理新的 K_j, V_j 块时, 先用本块分数 $s = Q_i K_j^T$ 更新最大值得到 $\tilde{m}$ (取 m 与本块最大值的较大者), 再把旧的累积量乘以 $\exp(m - \tilde{m})$ 缩放到新基准, 最后累加本块贡献:

$$\ell \leftarrow \exp(m - \tilde{m})\ell + \sum_j \exp(s_j - \tilde{m}) \quad (3.3)$$

$$O \leftarrow \exp(m - \tilde{m})O + \left(\sum_j \exp(s_j - \tilde{m})V_j \right) \quad (3.4)$$

更新后令 $m \leftarrow \tilde{m}$, 最终输出 $\frac{O}{\ell}$ 即为精确的 softmax 加权。关键在于: 当新块带来更大的最大值时, 旧的 ℓ 和 O 必须按 $\exp(m - \tilde{m})$ 重新缩放, 才能与新块在同一数值基准下相加。这保证了最终结果与标准 softmax 数学等价, 却从不需要物化整个 $N \times N$ 矩阵。

3.4.3 Ring Attention

Ring Attention (环形注意力) 是 FlashAttention 在多节点上的扩展。每个主机持有一个 query 块, KV 块沿环形拓扑在主机间传递。关键设计是**通信与计算重叠**: 当一个主机在计算注意力时, KV 块同时在网络上传递给下一个主机。这样通信开销被计算隐藏, 支持超长上下文的跨节点注意力。

3.4.4 线性注意力

线性注意力的核心思想是**重排矩阵乘法顺序**, 把 $O(N^2)$ 的注意力降到 $O(N)$ 。

标准注意力先算 $S = QK^T$ ($O(N^2d)$)，再算 $O = \text{softmax}(S)V$ 。如果把 softmax 替换为核函数 φ ，即用 $\varphi(q) \cdot \varphi(k)$ 近似 $\exp(q \cdot k)$ ，就可以**改变结合顺序**：

$$O = (\varphi(Q)\varphi(K)^T)V = \varphi(Q)(\varphi(K)^TV) \quad (3.5)$$

先算 $\varphi(K)^TV$ 得到一个 $d \times d$ 的状态 ($O(Nd^2)$ ，当 $d \ll N$ 时远小于 $O(N^2d)$)，再左乘 $\varphi(Q)$ 。复杂度降到 $O(N)$ 。

更关键的是，加上因果掩码后，线性注意力可以写成 **RNN 形式**：把整个历史压缩到一个常数大小的循环状态 S 中，每步只需更新状态和输出，不必回看历史 Token：

$$S_t = S_{t-1} + \varphi(k_t)v_t^T, \quad o_t = \varphi(q_t)S_t \quad (3.6)$$

状态 S 始终是 $d \times d$ 的固定大小，与序列长度 N 无关，因此每步开销相对 N 是 $O(1)$ 。这就是「常数大小状态」的由来：历史信息被不断累加进 S ，而非逐条保留。

变体	改进
GLA (Gated Linear Attention)	加入门控衰减，控制遗忘
DeltaNet	学习写什么到状态
GDN (Gated DeltaNet)	同时学习遗忘和写入
KDA, EDA, ...	更多变体

直觉 线性注意力就像一个有选择性遗忘的笔记本。标准注意力每一步都要翻看所有历史记录 ($O(N)$)，而线性注意力把历史压缩成一个固定大小的摘要（状态），每步只需更新摘要和生成输出 ($O(1)$)。代价是你不能精确回忆某个具体的旧 Token，只能依赖摘要中的信息。这就是为什么线性注意力常用于混合架构中处理长上下文部分。

FlashQLA 是 Qwen 团队为线性注意力开发的高性能内核，针对 GDN 做了硬件友好的重构：把门控衰减因子拆出来，降低 *Tensor Core* / *CUDA Core* / *SFU* 的开销；手工的 *warpgroup* 特化，重叠数据传输、*Tensor Core* 和 *CUDA Core* 的计算。前向 2-3 倍加速，反向 2 倍加速，支持门控感知的自动上下文并行。

线性注意力的 *prefix caching* 命中率极低（因为状态是压缩的，不像 *KV cache* 可以精确复用前缀），但它可能特别适用于 *Agentic* 场景：*Agent* 的长时程交互需要极长的上下文，线性注意力的 $O(N)$ 特性和常数大小状态正好匹配。

3.4.5 稀疏注意力

稀疏注意力的观察基础是：**注意力图天然稀疏的**。在长文本中，只有一小部分 Token 接收到大部分注意力权重（约 20% 的 Token 接收约 80% 的注意力）。如果只计算重要的 Token，就能大幅减少计算和 *KV cache* 访问。

稀疏注意力有三类：

1. **静态稀疏注意力** (Static Sparse Attention)：固定模式，如 **StreamingLLM**（保留开头的 sink Token 加滑动窗口的近期 Token，丢弃中间一切）和 **MIInference**。
2. **动态块稀疏注意力** (Dynamic Block Sparse)：自适应块选择，如 **SpargeAttn** 和 **XAttention**。

3. 动态 Token 稀疏注意力 (Dynamic Token Sparse): 细粒度 Token 选择, 如 DeepSeek Sparse Attention (DSA)。

静态模式 (如 StreamingLLM) 的洞察是: 注意力有注意力 sink 现象, 即开头的几个 Token 总是接收到大量注意力权重, 即使它们在语义上不重要。这是因为 softmax 需要一个锚点来分配剩余概率。保留这些 sink Token 加上近期窗口, 就能在极长上下文中保持稳定生成。

3.5 KV Cache 压缩

3.5.1 问题规模

KV cache 是长上下文推理的显存和带宽瓶颈。以 GLM-5.2 为例: 它是一个 7440 亿参数级别的 MoE (约 400 亿激活参数), 为长时程编码和研究 Agent 设计, 上下文窗口 1048576 个 Token (1M)。

例 GLM-5.2 在 1M Token 下的 KV cache 计算:

每 Token 的 cache 布局 (vLLM): 78 组 (512 维 MLA 潜向量 + 64 维 RoPE) BF16 加 21 个 DSA 索引缓存 (128 维 FP8 + scale) = 90.46 KiB/Token。

$$90.46 \frac{\text{KiB}}{\text{token}} \times 1048576 \text{ tokens} \approx 90.5 \text{ GiB}$$

这还只是一个 Agent。4 个独立的历史记录就是约 362 GiB, 还不包括权重和运行时缓冲区。长上下文 Agent 同时压垮 Prefill 和 Decode: Prefill 阶段一次性处理所有 prompt Token (计算密集), Decode 阶段每步都要重读整个 cache 并追加一个新条目 (访存密集, cache 每步增长)。

3.5.2 KV Cache 压缩分类

KV cache 压缩策略分为两大类:

1. **固定规则 (Static):** 预设的保留/驱逐规则, 不需要运行时注意力观察。
 - **StreamingLLM:** 保留开头的 sink Token 加滑动窗口的近期 Token, 驱逐中间的一切。
2. **基于注意力分数 (Dynamic):** 按历史注意力分数排序缓存 Token, 驱逐/不读最低分的。
 - **H2O:** 每步维护一个累积分数。
 - **ZipCache:** 用归一化累积注意力分数和探针 Token 动态排序。

直觉 静态策略简单但不灵活: 它不知道哪些 Token 真正重要, 只是机械地保留头部和尾部。动态策略更聪明但面临工程挑战: FlashAttention 隐藏了注意力分数 (不物化 S 矩阵), PagedAttention 打碎了内存 (碎片化使驱逐困难)。所以动态策略需要特殊设计才能与现有推理框架兼容。

3.5.3 三种 KV Cache 压缩方法

方法	核心思路	效果
----	------	----

ZipCache (NeurIPS 2024)	用归一化注意力权重评分缓存 Token 的显著性，兼容 FlashAttention。显著 Token 量化为 4 bit，其余 2 bit，混合精度	Mistral-7B/GSM8K 上 4.98 倍压缩，精度仅降 0.38%
MiniCache (NeurIPS 2024)	相邻的中后层持有几乎相同的 KV 状态，在深度维度合并：插值方向、保留幅度、保留异常 Token 不合并	最高 41% 显存减少，约 5 倍吞吐（vs FP16）
TriAttention (ICML 2026)	Q/K 向量在 pre-RoPE 空间中集中，注意力可以从几何预测，不需要观测分数。兼容 FlashAttention，在 PagedAttention 下压缩空闲块	2.5 倍吞吐，10.7 倍 KV 显存减少（同等精度）

MiniCache 已成为当前主流模型的标配。它的洞察很巧妙：不是在单层内压缩 KV，而是在层间合并。中后层的 KV 状态高度相似（因为模型已经在做类似的注意力计算），所以可以把相邻层的 KV 合并而不损失太多信息。这是一个全新的压缩维度：之前都在 Token 维度和精度维度压缩，*MiniCache* 开辟了深度维度。

TriAttention 的实际效果可以用一个 Demo 说明：任务是读取六个项目文件并写周报，模型是 Qwen3-32B-INT4 (AWQ)，运行在单张 RTX 4090 (24GB) 上。没有 *TriAttention* 时，KV cache 在任务中途耗尽导致 OOM；有 *TriAttention* 时，在显存预算内完成任务。

3.5.4 分层 KV Cache 系统

长上下文的 KV cache 远超 GPU HBM 容量时，需要像操作系统的内存层次一样做分层缓存：

层级	容量	带宽	角色
GPU HBM	几十 GB	≈ TB/s	最热，活跃生成
CPU DRAM	几百 GB 到 TB	PCIe ≈ 64 GB/s (HBM 的 2%)	温，最近被驱逐的
本地 SSD/NVMe	几十 TB	几 GB/s	冷，超出 DRAM 的溢出
远端/分布式存储	近乎无限	网络受限 RDMA	跨实例/会话共享

例 延迟差异巨大：移动 50 GB 的 KV cache，从 HBM 只需约 15 ms，但从 CPU DRAM 需要约 800 ms，差 53 倍。系统如 **NVIDIA Dynamo** 会根据上下文增长自动把冷的 KV 块从 HBM 卸载到 DRAM，再到 SSD，再到远端存储。

在 Agent 场景中，这种分层尤为关键：**热记忆**（hot memory）是活跃会话的近期对话和工具输出，保持在 HBM 中供即时复用；**冷记忆**（cold memory）是较早的对话和长期事实，推送到 SSD 或远端，只在相关时才取回。

3.5.5 Hy-Memory 案例

Hy-Memory（腾讯混元）是 OpenClaw Agent 框架的长期 Agent 记忆系统。它要解决的问题是三周轨迹现象：Agent 在第一周表现热情（能力在线），第二周开始因上下文遗忘而变得沮丧，第三周退化为只能回答简单查询。

架构设计采用双层系统：System 1（实时）处理 L1 到 L4 的快速记忆，System 2（异步，秒到分钟级）蒸馏 L5 到 L6 的长期记忆而不阻塞对话。效果：在 LongMemEval 基准上得分 85.2，处理长上下文输入时使用 35% 更少的 Token，内存密度比同类方法高 3 到 4 倍，写入速度比 Graphiti 快 8 倍。

3.6 KV Cache 与注意力架构协同设计

KV cache 的大小直接由 K/V 的设计决定。从 MHA 到 MLA，架构师们在效果与显存之间做了极限拉扯：

设计	K/V 设计	每 Token 每层缓存	效果与显存	代表模型
MHA	每个 head 独立一组 K, V	$2 \cdot h \cdot d_k$ （最大）	效果最好，缓存最大	Transformer, LLaMA2-7B
MQA	全部 h 个 head 共享 1 组 K, V	$2 \cdot d_k$ ，为 MHA 的 $\frac{1}{h}$	压缩到极限，效果有损	PaLM, StarCoder, Gemini
GQA	每 $\frac{h}{g}$ 个 head 一组，共 g 组 ($1 < g < h$)	$2 \cdot g \cdot d_k$ ，为 MHA 的 $\frac{g}{h}$	两者折中；70B 常取 $g = 8$ 匹配单机 8 卡	LLaMA2/3-70B, Yi, ChatGLM2/3
MLA	K, V 低秩联合压缩为潜向量 c_t ，再升维恢复各 head	$d_c + d_r = 512 + 64$ ，与 h 无关	缓存 $\approx$ MQA，效果 $\approx$ MHA；RoPE 需解耦 d_r 维单独加	DeepSeek-V3

直觉 MHA 是每个 head 有自己的 K/V 笔记本，效果最好但缓存最大。MQA 走极端：所有 head 共用一个笔记本，缓存最小但效果有损。GQA 是折中：几个 head 共用一个笔记本。MLA 最巧妙：不直接共享，而是把 K/V 低秩压缩成一个潜向量 c_t ，推理时从这个潜向量升维恢复各 head 的 K/V，恒等变换为 MQA 形式。这样缓存大小与 head 数 h 无关（只取决于压缩维度 $d_c + d_r = 576$ ），但效果接近 MHA。

MLA 的技术细节：K/V 被联合压缩为 $c_t = W^{KV} h_t$ （一个低维潜向量），推理时缓存 c_t 。每个 head 的 K 和 V 从 c_t 升维恢复： $K_i = W_i^K c_t$, $V_i = W_i^V c_t$ 。RoPE 无法在压缩空间中直接应用（因为 RoPE 是逐位置旋转，与低秩压缩不兼容），所以需要解耦一部分维度 d_r 单独做 RoPE。这就是为什么 MLA 的缓存是 $d_c + d_r = 512 + 64 = 576$ 。DeepSeek-V4 进一步发展了 CSA（Compressed Sparse Attention）。

3.6.1 面向世界模型的注意力：WorldAttention

把视野从语言模型推向**世界模型**（world model，用于视频生成与交互式模拟），KV cache 与注意力的协同设计面临更极端的矛盾：交互要求保留长历史以维持一致性，而全历史 KV cache 又会迅速撑爆显存。**WorldAttention** 用系统-算法协同设计应对这一矛盾，包含两个组件：

1. **分层 KV Cache (HKV)**：用粗到细的分页内存检索，把 KV cache 组织成层级结构，按需取出相关块，既保留长历史又控制显存。
2. **混合稀疏注意力 (HSA)**：双分支的高效注意力计算，对检索回来的 KV 做稀疏化处理，去除冗余 Token 并缓解碎片化访存。

直觉 世界模型的注意力瓶颈是两方面的：一方面，滑动窗口 KV cache 会遗忘早期帧导致主体不一致，而全历史 KV cache 又超出显存；另一方面，检索回来的 KV 中包含大量冗余 Token 且访存碎片化，使解码开销大、硬件利用率低。HKV 解决「留多少」的问题（分层缓存），HSA 解决「算多少」的问题（稀疏计算）。两者协同后，WorldAttention 在交互式视频生成中保持了更强的主体一致性、更平滑的运动过渡和更稳定的场景结构。

WorldAttention 是本章主线的一个缩影：KV cache 压缩（HKV）加高效注意力（HSA）协同，把「分层存储」与「稀疏计算」两套思路合在一起，应对世界模型「既要长记忆、又要低开销」的需求。这与前面分层 KV cache 系统、稀疏注意力、MLA 低秩压缩的思路一脉相承。

3.7 高效解码

3.7.1 投机解码

自回归解码每步只生成一个 Token，而且 Decode 阶段是访存密集的：每步都要重读整个 KV cache。**投机解码**（Speculative Decoding）的洞察是：一个小的**草稿模型**（draft model）可以快速生成 K 个候选 Token，然后大的**目标模型**（target model）一次性并行验证所有候选位置。

算法如下：

输入：前缀 x ，草稿模型 q ，目标模型 p ，前瞻 K

while not EOS:

```
# 1. 草稿模型串行生成 K 个 token
 $y_1 \dots y_K \sim q(. | x)$ 
# 2. 目标模型并行验证所有候选位置
compute  $p(. | x, y_{<1}), \dots, p(. | x, y_{<K})$ 
# 3. 接受或拒绝
for  $i = 1 \dots K$ :
     $\alpha = \min(1, p(y_i) / q(y_i))$ 
    if Uniform(0,1)  $\leq \alpha$ :
        append  $y_i$  to  $x$ 
    else:
        sample  $z \sim \text{normalize}(\max(p - q, 0))$ 
        append  $z$  to  $x$ 
        break
```

```
# 4. 如果全部接受, 额外采样一个目标 token
if all accepted:
    sample z ~ p(. | x, y1...yK)
    append z to x
```

直觉 投机解码的精妙之处在于：草稿模型串行生成（慢但可以并行验证），目标模型并行评分（快，因为 Prefill 是计算密集的）。接受/拒绝的阈值 $\alpha = \min\left(1, \frac{p(y_i)}{q(y_i)}\right)$ 保证了最终输出的分布与目标模型完全一致，没有任何精度损失。如果草稿模型猜对了（ $p(y_i)$ 和 $q(y_i)$ 接近），就接受；如果猜错了（ $p(y_i)$ 远大于 $q(y_i)$ ），就拒绝并从修正分布中重采样。

投机解码的局限：需要维护一个独立的草稿模型，增加了显存和复杂度；接受 Token 数是变量，会打乱常规的批处理调度。但它的收益是巨大的：在理想情况下，一次前向传播可以生成 $K+1$ 个 Token 而不是 1 个。

3.7.2 多 Token 预测 (MTP)

Medusa / EAGLE 系列不需要额外的草稿模型，而是在目标模型上添加并行的解码头。这些头预测多个候选 Token，top-k 输出形成树状候选续写。目标模型在一次批量前向传播中验证多个候选位置。

1. **Medusa**：最简单的方案，添加多个并行解码头，每个头独立预测一个未来位置。
2. **EAGLE-3**：融合多层目标特征，直接预测草稿 Token，准确率更高。
3. 更多候选可以增加接受 Token 数，但也增加了验证工作量（变成计算密集）。

3.7.3 Diffusion-LLM 的 MTP

一个更前沿的方向是用**扩散模型**来做草稿生成：

1. **DFlash**（块扩散并行起草）：目标特征到掩码块到一次提议到验证。重点是最小化草稿延迟。
2. **DSPark**（依赖感知起草加调度验证）：并行骨干到串行块到置信度前缀到验证。
DSPark 只在草稿模型有信心位置做验证，减少浪费的验证计算。

例 实测效果：DFlash 在不同任务上达到 2.75 到 6.08 倍加速；DSPark 在匹配吞吐下每用户速度提升 60% 到 85%。DSPark 的关键创新是在高负载下仍然有效：不是验证每个草稿 Token，而是只在草稿模型有信心的地方验证，这样即使 batch size 很大也能保持加速。

3.7.4 K-Forcing

K-Forcing 解决的是投机解码的一个结构性问题：接受 Token 数是变量，会打乱常规批处理。K-Forcing 用一个**推前语言模型**（push-forward language model）把独立同分布的均匀噪声 Token 映射为固定长度的未来 Token 块，建模它们的联合分布（而非像掩码语言模型那样只建模边际分布）。

各种解码加速方法的权衡：投机解码需要额外草稿模型但保证分布一致；MTP 用并行头避免额外模型但候选树调度复杂；Diffusion-LLM 草稿质量高但引入扩散采样开销；K-

Forcing 保证固定长度输出但需要新的模型架构。没有银弹，选择取决于延迟目标、吞吐需求和系统复杂度。

3.8 推理框架

现代推理框架的几项关键技术反复出现，值得先理解清楚：

1. **PagedAttention**：把 KV cache 切成固定大小的页（page），按需分配，像操作系统的虚拟内存分页一样管理显存。它消除了为每个请求预连续分配整块显存造成的碎片，让显存利用率大幅提升，也使缓存块可以在请求间共享。
2. **连续批处理**（Continuous Batching / In-flight Batching）：传统批处理要等一批请求都完成才能接新请求，长尾请求拖慢整批；连续批处理则在每一步动态地把新请求加入、完成的请求移出，让 GPU 始终被填满。
3. **前缀缓存**（Prefix Cache）：多个请求共享相同前缀（如系统提示词）时，只算一次前缀的 KV 并缓存复用，避免重复计算。
4. **RadixAttention**：SGLang 的核心，用一棵基数树（radix tree）组织前缀缓存，自动识别不同请求间的公共前缀并共享 KV，使结构化输出与前缀复用的命中率最大化。

直觉 这几项技术分别攻克推理的不同痛点：PagedAttention 攻克显存碎片，连续批处理攻克 GPU 空闲，前缀缓存和 RadixAttention 攻克重复计算。理解了它们，再看各框架的差异就清楚了：vLLM 以 PagedAttention 加连续批处理为根基走通用生态；SGLang 用 RadixAttention 把前缀复用做到极致以服务低延迟高吞吐；KTransformers 针对显存不足把 MoE 专家放到 CPU；TensorRT-LLM 则把 NVIDIA 硬件的融合内核堆到最满。

框架	主要聚焦	关键技术	最佳场景
SGLang	生产服务	RadixAttention；前缀缓存；结构化输出；投机解码	低延迟/高吞吐 API 和集群
vLLM	通用服务	PagedAttention；连续批处理；前缀缓存；投机解码	广泛的模型和硬件生态
FlashInfer	GPU 内核库	Attention, paged KV, GEMM, 量化和 MoE 内核	自定义服务栈和内核后端
KTransformers	CPU-GPU 混合 MoE	CPU 专家内核；offload/调度；量化	显存有限的稀疏大模型
TensorRT-LLM	NVIDIA 推理运行时	in-flight batching；paged KV；量化；多 GPU；投机解码	NVIDIA 优化的生产部署

选择框架不是看谁最快，而是看模型结构、硬件、并发、延迟目标和工作负载。vLLM 生态最广适合通用场景；SGLang 在低延迟和高吞吐上优化最深；KTransformers 适合 GPU 显存不够但 CPU 内存大的 MoE 模型；TensorRT-LLM 在 NVIDIA 硬件上极致优化。

3.9 分离部署

推理的不同阶段和不同模块有截然不同的特性：

1. **Prefill vs Decode**: Prefill 是计算密集的，Decode 是访存密集的。
2. **Attention vs FFN**: Attention 有大量内存访问，FFN 是纯计算。
3. **不同 GPU**: H20 带宽富裕但算力弱（适合 Decode），H100 算力强但带宽相对弱（适合 Prefill）。

基于这些差异，可以做两种分离部署：

1. **PD 分离** (PD Disaggregation): Prefill 和 Decode 部署在不同的 GPU 实例上。Prefill 实例用算力强的 GPU，Decode 实例用带宽富裕的 GPU。
2. **AF 分离** (AF Disaggregation): Attention 和 FFN 部署在不同的 GPU 实例上，各自优化。

直觉 分离部署的本质是术业有专攻。就像工厂里，不同的工序在不同的车间完成，每个车间配置最适合的设备。Prefill 车间需要强大的压机（算力），Decode 车间需要宽大的传送带（带宽）。混在一起做时，两种资源都无法充分利用。

3.10 多模态与视频生成推理加速

多模态推理分两大方向：

3.10.1 理解方向 (Understanding)

瓶颈在 Prefill 注意力和 Decode 访存。**ZipVL** (ICCV 2025) 通过高效近似 KV 重要性来加速：Prefill 阶段跳过不重要的计算，Decode 阶段不加载不重要的 KV Token。

3.10.2 生成方向 (Generation)

瓶颈在重复的大规模计算（扩散模型的迭代采样）。加速手段包括：

1. **减少去噪步数**: VideoLCM、T2V-Turbo、DOLLAR。
2. **跨去噪步复用计算**: TeaCache、FasterCache。
3. **稀疏/线性注意力**: SVG、BLADE (ICLR 2026)、PSA (CVPR 2026)。

3.10.3 视频生成基础设施

1. **DAX** (Diffusion Accelerated eXecution): 高性能扩散模型推理引擎。技术包括 FP8/INT8 GEMM 和 SageAttention2 量化、序列并行加通信重叠、TeaCache 跳过平凡去噪步、torch.compile 融合量化和通信算子。
2. **SGLang Diffusion**: 高性能图像和视频生成推理框架。支持多种注意力后端 (FlashAttention、SageAttention、多种稀疏注意力)，FP8/NVFP4/INT4 量化和 Cache-DiT/TeaCache，Ulysses/Ring 序列并行和 TP/FSDP/CFG 并行。
3. **TurboDiffusion**: 模型-系统协同设计加速框架 (100-200 倍加速)。W8A8 量化，rCM 步蒸馏把扩散采样减少到 1-4 步，SageSLA 结合低比特 SageAttention 和可训练稀疏线性注意力，融合归一化和量化 GEMM 内核。

3.10.4 Block Diffusion 与 Inferix

Block Diffusion 在自回归和扩散之间插值：不是逐 Token 也不是全序列扩散，而是按块做扩散生成。**Inferix** 是一个面向世界模拟的推理引擎，支持高级 KV cache 管理、分布式世界

合成、交互式生成（暂停/恢复/即时改 prompt）、视频流（RTMP/WebRTC）、量化推理（8 bit），甚至能在 16 GB 消费级 GPU 上运行。

4 第三章：Agent Loop

AI 编码的演进路线是：2020-22 年的自动补全（ghost text）→ 2023 年的 RAG 上下文 → 2024 年的 Agent 转向（能行动的工具）→ 2025 年的推理和多步 Agent → 2026 年的自主运营。从工单到合并 PR，并行 Agent 像团队一样分工、委派和验证任务。我们正处于 Software AGI 的黎明。

4.1 什么是 Agent Loop

Agent Loop（Agent 循环）是一个程序，它反复让模型执行以下循环，直到目标达成或停止条件触发：

1. **目标与状态**（Goal + state）：定义目标和预算。
2. **规划**（Plan）：决定下一步做什么。
3. **行动**（Act）：调用工具、代码、shell。
4. **观察**（Observe）：获取输出和错误。
5. **验证**（Verify）：检查测试和验收标准。
6. 如果失败或不完整，回到规划。

直觉 Agent Loop 和普通的函数调用有什么区别？关键在于**持久状态**（durable state）。文件、记忆、实验记录、检查点让迭代能够超越单个上下文窗口的累积。人类的角色从每步都参与转变为定义目标、权限、预算和停止条件。

4.2 AutoResearch：自然语言驱动的研究闭环

Karpathy 的 **AutoResearch** 是 Agent Loop 的极致案例：整个框架就是一份自然语言 prompt（program.md）。

1. 一个 LLM 充当夜间研究实习生：修改 train.py 加入一个实验想法 → 运行 → 读取 val_bpb（验证集 loss）。
2. **可验证指标作为 reward**：如果 loss 改善就保留 commit，否则 git reset。每个想法被同一个数字评判。
3. **NEVER STOP**：不停下来问人；没想法了就读论文、组合接近成功的尝试、尝试更激进的变化。
4. 7 × 24 在 GPU 节点上运行，直到手动中断。这是 vibe research，超越了 vibe coding。

AutoResearch 的哲学很纯粹：把科学方法（假设、实验、验证）编码成一个循环，让 Agent 7 × 24 不间断地执行。关键是 reward 必须是可验证的（一个客观的数字），而不是主观判断。这样 Agent 才能自主判断实验是否成功。

4.3 启发式学习与持续学习

启发式学习（Heuristic Learning, HL）或**持续学习**（Continual Learning）提出了一种不依赖梯度的学习范式：编码 Agent 直接修改代码来改进策略，权重就是代码本身。

维度	深度强化学习	启发式学习 (HL)
Policy	神经网络参数	代码：规则、状态机、控制器、MPC、宏动作
State	通常为显式观测	显式变量、检测器、缓存，可读的表示
Action	一次神经网络前向传播	执行代码逻辑
Feedback	主要是固定奖励	测试、环境反馈、日志、回放、人工输入
Update	基于梯度更新神经网络参数	编码 Agent 直接修改代码
Memory	同策略：极少；异策略：经验回放缓冲区	试验、摘要、失败原因、回放、版本差异

直觉 深度 RL 的 policy 是一个黑盒神经网络，更新靠梯度；HL 的 policy 是一段可读的代码，更新靠 Agent 直接改代码。核心转变是：学习变成了维护一个软件系统，有明确的更新路径。一个健康的启发式系统要平衡三个职责：吸收反馈（失败、日志、奖励、人工纠正），保持能力（回归测试、固定种子回放、黄金轨迹），压缩历史（把局部补丁重构为更简洁的代码）。

HL 相对于深度 RL 的优势：可解释性（代码逻辑可读），过拟合控制（多种子评估），样本效率（一次代码更新可以带来巨大提升），减少遗忘（能力持久化在规则和测试中），可回归测试（测试、回放、黄金用例）。遗忘变成了工程问题：测试加回放加重构，而不是参数技巧。

4.4 自我改进的 Harness

Self-Harness（自改进外壳）是一个提议、评估、接受循环：

1. **弱点挖掘**：把失败聚类为有验证器支撑的根因模式。
2. **Harness 提议**：在可编辑表面做有界修改，针对反复出现的、可寻址的模式。
3. **验证**：只接受在 held-in 和 held-out 上都无回归的修改。

关键约束：权限和安全必须在循环外部，*reward hacking* 仍然是一个风险。*Harness* 本身变成了一个学习到的制品。这是一个元级别的循环：不仅模型在改进，编排模型的 *Harness* 也在自我改进。

4.5 Agent-as-a-Router

Agent-as-a-Router 是一个实际应用案例：用 Agentic 路由来分配编码任务。在 3000 个编码任务上测试，Agentic Router 框架的平均任务得分高于 Opus（最贵模型），而成本不到 Opus 的一半。核心思想是：不是所有任务都需要最强的模型，Agent 可以根据任务难度路由到不同能力的模型。

4.6 Kernel 优化 Agent

Agent Loop 在 HPC 内核优化领域展现了惊人的威力。一次性 LLM 在 KernelBench (250 个任务) 上只击败 PyTorch eager 的不到 20%，真正的突破来自循环：

1. **KernelFalcon** (PyTorch)：在所有 250 个 KernelBench 任务上达到 100% 正确率。
2. **KernelPro**：Level 3 上 5.30 倍几何平均加速，匹配速度下能耗降低 11.6%。
3. **KernelEvolve** (Meta, ISCA 2026)：生产中推理吞吐提升 60%。
4. **KDA** (Kernel Design Agents, NVIDIA)：在 MLSys'26 FlashInfer Track 上 Top-3, DSA indexer 加速 1.37 到 19.08 倍，DSA attention 加速 6.15 到 15.22 倍，FP8 MoE 从 0.27 到 1.31 倍。

直觉 Kernel Agent 的循环是：正确性验证门 → 基准测试 → NCU profile → 有针对性的下一步修改。这不是随机试错，而是基于 profiler 证据的测量搜索。瓶颈在哪 (stall、吞吐量、时间线)，下一步就改哪里。

KDA 的成功不是单一技巧，而是结构、知识和 profiler 证据的复合：Humanize loop 负责任务控制 (计划、执行、验证、审查)，KernelWiki 提供知识 (来自 SGLang/vLLM/TRT-LLM/DeepGEMM 的 PR)，ncu-report-skill 提供证据 (stall、吞吐量、时间线)。三者复合达到 6.26 倍加速。值得注意的是 GDN decode 上 0.92 倍 (没有加速)，这是基于验证器事实的诚实报告。

5 第四章：前沿与展望

5.1 多模态基础模型的演进

多模态模型经历了三个阶段：

1. **VLM** (视觉语言模型，理解)：如 **LLaVA**、**Qwen-VL**。把视觉编码器通过视觉指令微调挂接到 LLM 上，输出只有文本。感知和推理，但不能生成图像。
2. **统一理解与生成**：如 **Show-o**、**Janus-Pro**。一个 Transformer 同时做 AR 文本和 (离散) 扩散图像。Janus-Pro 为理解和生成两个角色解耦视觉编码。
3. **图像/视频生成**：如 **Wan 2.x**、**HunyuanVideo**。扩散 Transformer (DiT) 作为视频基础模型。视频是世界的一次 rollout：预测世界如何演化。开源视频模型中 Wan 2.7 领跑 Wan-Bench 2.0，HunyuanVideo 1.5 可在单张 4090 上运行。

直觉 这条演进路线的本质是：从看懂图像到生成图像再到模拟世界如何演化。每一步都在缩小模型与物理世界之间的鸿沟。生成不是目的，模拟才是：能够预测下一帧图像，就意味着模型理解了物理规律和因果关系。

5.2 从 VLA 到世界动作模型

在视觉语言模型的基础上加入行动维度，就进入了具身智能的领域：

1. **VLA** (Vision-Language-Action)：如 $\pi-0.5$ 、**Qwen-VLA**。观测加指令到 VLM 骨干到动作专家 (流/扩散) 到机器人动作。 $\pi-0.5$ 实现未见环境的泛化；Qwen-VLA 用统一动作空间控制多种机器人本体。

2. **世界动作模型**：如 **LingBot-VA**。交错序列 $v_1 a_1 v_2 a_2 \dots$ ，因果视频-动作世界模型。先想后做：想象下一帧再行动。在 LIBERO-Long 上 98.5%，30-50 个 demo 即可适应新任务。
3. **统一动作模型**：如 **Cosmos-3**、 $\pi - 0.7$ 。一个全能 Transformer（推理加生成加动作），双塔 MoT 吞噬 VLM、视频生成器、模拟器和策略。 $\pi - 0.7$ 是可操控的通才，有涌现的技能组合。

动作闭环了循环：从预测世界到在世界中行动。这是从旁观者到参与者的根本转变。一个能想象下一帧并据此行动的模型，本质上在做心理模拟：先在脑中预演动作的后果，再选择最优行动。

5.3 4D 世界模型

未来的方向是 4D 世界模型：实时、可交互、物理可信、几何一致、长时程、高保真。为什么需要 4D 世界模型？它是 Agentic AI、具身 AI 和游戏的模拟器。扩大世界模型的规模会涌现出理解（感知和推理）能力。LLM 为中心的时代将会过去，视觉为中心才是未来。

5.4 Agentic 世界模型

Agentic 世界模型的核心是一个数据飞轮：

1. **Agent 场景合成**：Agent 组合 3D 场景、资产和任务规格。
2. **模拟 rollout**：具身轨迹大规模生成。
3. **环境反馈**：成功/失败信号、物理违规。
4. **模型迭代**：重训世界模型和策略。

例 具身数据是瓶颈：真实机器人采集慢，模拟场景手工搭建。Agentic 方法的解法是 Agent 合成可模拟的场景和任务，环境反馈过滤和改进它们。每次循环迭代都同时改进数据和模型。合成的场景成本比商业场景管线低约 100 倍。

5.5 产业级基础设施挑战

5.5.1 训练侧

1. **万卡集群动态容错**：避免全集群重启，实现精度无损的故障恢复。
2. **长序列通算融合 Attention**：挖掘 Blackwell 新架构算力并实现高效的通信-计算重叠，提升长序列下的 MFU。
3. **多模态与短序列优化**：支持任意 Mask 的稀疏计算与负载均衡，弥补开源 FlashAttention 在 B 卡短序列场景的性能差距。
4. **Data/Training Server 服务化架构**：数据预处理去中心化，支持多用户 LoRA 任务在线调度。

5.5.2 推理侧

1. **多级 KV Cache 与存储优化**：构建 HBM-SSD-远端存储四级缓存并结合 cache 压缩，在保持约 90% 缓存命中率的同时大幅压降存储成本。
2. **异构计算统一图优化**：自动构图与算子融合替代低效的人工调优。
3. **多模态无损推理**：低比特量化与投机解码须保证生成质量零回退。

4. **动态资源调度与新硬件适配**：以负载感知的动态 PD 调度应对训推波峰波谷，并针对 Blackwell 显存小、通信弱的特点实现计算-通信重叠。

5.6 面向 Agent 的 AI 芯片

面向 Agent 的 AI 芯片需要：高带宽存储与大规模 AI 计算能力、大模型训练与推理系统、AI 编译器及自动优化、高性能 Kernel、面向新型芯片架构的软硬件协同优化（如 3D 堆叠、光通信、原生多级存储等）。

6 本章你将学会

1. 解释 Token 经济和 Scaling Law 如何驱动算法与系统协同设计，以及效率四要素和 MLSys 三角的含义。
2. 描述训练数据流水线各阶段，说明 AdamW 和 Muon 优化器的原理差异，以及 MoE 与混合注意力架构的设计动机。
3. 比较五种并行策略（DP/TP/PP/EP/CP）的切分对象、优缺点和适用场景，以及 LoRA/QLoRA 如何实现参数高效微调。
4. 区分 SFT、Offline RL、Online RL 和 OPD 四种后训练方法的数据来源、训练信号和 On-policy 特性。
5. 说明 RL 基础设施（VeRL/Slime 等）的组件分工，以及 Agentic RL 的分布式系统挑战。
6. 解释推理 Prefill 与 Decode 的瓶颈差异，量化格式（Binary/INT/FP）的区别，以及 NVFP4 两级缩放为何让 4 bit 近乎无损。
7. 描述异常值问题及 LLM.int8()、SmoothQuant、QuaRot/SpinQuant 的解决思路。
8. 阐述 FlashAttention 通过分块和在线 softmax 减少 HBM 访存的原理，以及线性注意力和稀疏注意力的设计思想。
9. 说明 KV cache 压缩的分类（静态/动态）、三种方法（ZipCache/MiniCache/TriAttention）的原理，以及分层 KV cache 系统的设计。
10. 比较从 MHA 到 MLA 的四种 K/V 设计在缓存大小和效果上的权衡。
11. 描述投机解码、MTP、Diffusion-LLM MTP 和 K-Forcing 的原理与权衡。
12. 说明推理框架的关键技术（PagedAttention、连续批处理、前缀缓存、RadixAttention）如何分别解决显存碎片、GPU 空闲和重复计算，以及 PD/AF 分离部署的动机。
13. 解释 Agent Loop 的定义、AutoResearch 的设计理念，以及启发式学习与传统深度 RL 的区别。
14. 说明 Kernel 优化 Agent 的循环机制和 KDA 的成功要素。
15. 描述多模态基础模型从 VLM 到世界模型的演进路线，以及 4D 世界模型和 Agentic 数据飞轮的愿景。

7 要点速查

概念	英文	一句话要点
Token 经济	Token Mxing	吞吐和 Token 效率是新的护城河，不是模型数量

预训练 Scaling Law	Pre-training Scaling Law	训练算力与 loss 的幂律关系，高质量数据趋于耗尽
测试时 Scaling Law	Test-time Scaling	推理时更多思考 Token 可提升效果，log 关系
数据飞轮	Data Flywheel	Agent 交互产生数据，数据训练模型，模型部署为更强 Agent
MLSys 三角	Compute/Memory/Communication	三者必须协同扩展，任一瓶颈卡住整个系统
Agent	Agent = Model + Harness	Harness 是编排代码，白盒快迭代；模型是黑盒慢更新
MoE	Mixture of Experts	总参数大但每 Token 只激活小部分，如 671B/37B
AdamW	AdamW	逐参数自适应步长，默认优化器，鲁棒跨规模
Muon	Muon	矩阵感知优化，Newton-Schulz 正交化，约 2 倍效率
LoRA	Low-Rank Adaptation	冻结 W_0 ，训练低秩 BA ，参数减少万倍
QLoRA	QLoRA	在 4 bit 量化基座上做 LoRA，单 48GB GPU 微调 65B
FSDP2	PyTorch FSDP2	原生全分片，ZeRO-3 式，基于 DTensor
SFT	Supervised Fine-Tuning	外部数据集，Forward KL，高遗忘低成本
Online RL	PPO/GRPO	当前 policy rollout，Reverse KL，低遗忘高成本
OPD	On-Policy Distillation	学生采样，teacher logits，Reverse KL
Slime	Slime	Megatron + SGLang + buffer，极简 RL 框架
NVFP4	NVFP4	4 bit 浮点，两级缩放，3 倍吞吐，近乎无损
SmoothQuant	SmoothQuant	把激活异常值转移到权重上再量化
QuaRot/SpinQuant	Rotary Quantization	正交旋转分散异常值，旋转不变性
FlashAttention	FlashAttention	分块加在线 softmax，不物化 $N \times N$ ，减少 HBM 访问
线性注意力	Linear Attention	重排矩阵乘法到 $O(N)$ ，RNN 形式常数状态
稀疏注意力	Sparse Attention	只算重要 Token， $\approx O(Nk)$
StreamingLLM	StreamingLLM	保留 sink Token 加滑动窗口，驱逐中间

ZipCache	ZipCache	归一化注意力评分，混合精度 4/2 bit，4.98 倍压缩
MiniCache	MiniCache	跨层合并相似 KV，深度维度压缩，主流标配
TriAttention	TriAttention	几何预测注意力，不需观测分数，10.7 倍 KV 压缩
分层 KV Cache	Hierarchical KV Cache	HBM 到 DRAM 到 SSD 到远端
MHA/MQA/GQA/MLA	Attention K/V Design	从独立到共享到低秩压缩，MLA 缓存近似 MQA 效果近似 MHA
投机解码	Speculative Decoding	草稿模型串行生成，目标模型并行验证，分布一致
MTP	Multiple-Token Prediction	并行解码头，树候选，一次验证多个位置
PD 分离	PD Disaggregation	Prefill 和 Decode 部署到不同 GPU 实例
Prefill/Decode 瓶颈	Prefill vs Decode	Prefill 计算密集、Decode 访存密集，是推理优化的出发点
PagedAttention	PagedAttention	KV cache 分页管理显存，消除碎片并支持块共享
RadixAttention	RadixAttention	基数树组织前缀缓存，自动共享公共前缀
Agent Loop	Agent Loop	Plan-Act-Observe-Verify 循环，持久状态
AutoResearch	AutoResearch	自然语言 prompt 驱动 7x24 研究闭环
启发式学习	Heuristic Learning	Agent 改代码即改 policy，遗忘变工程问题
Kernel Agent	KernelPro/KDA	正确性到基准到 profile 到修改循环
4D 世界模型	4D World Model	实时可交互物理可信的模拟器，视觉为中心

8 小结

本章从 Token 经济和 Scaling Law 出发，系统梳理了大模型从训练到推理到 Agent Loop 的全链路。我们看到了效率优化是一个多层次的问题：数据层面有流水线压缩和合成数据；算法层面有 MoE、线性注意力、量化等架构创新；系统层面有 FlashAttention 的 IO 感知设计、KV cache 的分层压缩、PD 分离部署等基础设施协同。

训练系统的核心挑战是切分：如何把大模型切分到数千张 GPU 上（DP/TP/PP/EP/CP），以及如何用更好的优化器（Muon）和参数高效微调（LoRA）来降低成本。后训练从 SFT 到 RL 再到 OPD，On-policy 数据是防止遗忘的关键变量。RL 基础设施（VeRL/Slime）需要协调训练引擎和推理引擎的异步协作，Agentic RL 更是分布式系统问题。

推理系统的核心洞察是 Prefill 计算密集与 Decode 访存密集的本质差异。低比特量化从 INT8 到 NVFP4，两级缩放让 4 bit 近乎无损。注意力优化三大家族各有定位：FlashAttention 让精确注意力不再被 I/O 卡住，线性注意力把复杂度降到 $O(N)$ ，稀疏注意力用智能选择逼近全注意力效果。KV cache 压缩从 Token 维度（ZipCache）到深度维度（MiniCache）到几何预测（TriAttention），配合分层存储系统支持百万 Token 上下文。从 MHA 到 MLA 的架构演进，用低秩压缩实现了 MQA 级缓存和 MHA 级效果。

Agent Loop 是连接算法和真实世界的桥梁。AutoResearch 展示了自然语言 prompt 驱动的 7×24 研究闭环；启发式学习把学习从梯度更新转变为代码维护；Kernel Agent 用正确性、基准、profile、修改的循环在 HPC 内核优化上达到惊人效果。

展望未来，多模态模型从理解到生成到模拟世界，VLA 让模型从旁观者变成参与者，4D 世界模型和 Agentic 数据飞轮指向一个视觉为中心的未来。产业级基础设施仍面临万卡容错、长序列通算融合、多级 KV cache、异构图优化等挑战。

一句话总结：**算法创新加基础设施协同加 Agent 闭环 = 通向 AGI 的路径。**

讲义基于 HPC101 课程 Bohan Zhuang 「Algorithm-Infra Co-Design for AGI」内容编写